1. Introduction to AI for Software Testers

1. What is Artificial Intelligence ?

Artificial Intelligence (AI) is the ability of machines to simulate human intelligence and perform tasks that typically require human thinking, learning and decision making.

2. Why AI is Important for Testers ?

- → Makes testing faster and smarter.
- → Helps in test case creation, test data generation and bug analysis.
- → Improves test coverage and accuracy.
- → Enables testers to work on high value testing tasks.
- → Future of quality engineering is AI-driven.

3. AI vs ML vs Deep Learning vs Generative AI

AI	Umbrella term for machines that can think and act intelligently.
ML	Subset of AI that learns from data and improves with experience.
Deep Learning	Subset of ML that uses neural networks with multiple layers.
Generative AI	Subset of AI that can generate new content like text, images, code, etc.

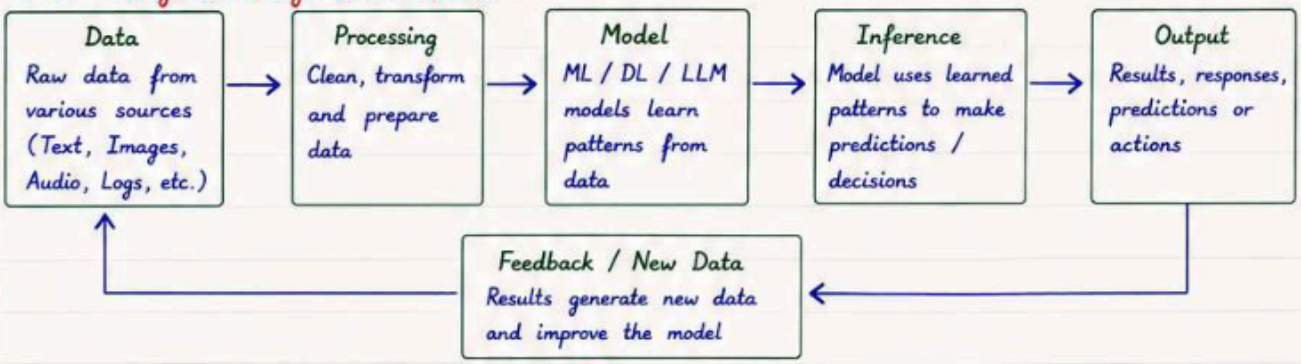
4. Traditional Software vs AI-based Software

Aspect	Traditional Software	AI-based Software
Logic	Explicit rules written by developers	Learns patterns from data
Adaptability	Fixed until code is changed	Improves with more data
Decision Making	Deterministic (if-else, rules)	Probabilistic (based on prediction)
Data Dependency	Low	High
Examples	Calculator, Billing System, CRUD apps	ChatGPT, Recommendation Engines, Self-driving cars

5. Real-world AI Applications

- → Virtual Assistants (ChatGPT, Siri, Alexa)
- → Recommendation Systems (Amazon, Netflix, YouTube)
- → Fraud Detection in Banking
- → Self-driving Cars
- → Healthcare Diagnosis
- → Image / Face Recognition
- → Spam Detection in Emails
- → Predictive Analytics
- → Smart Automation in Testing

6. AI Ecosystem (High Level View)



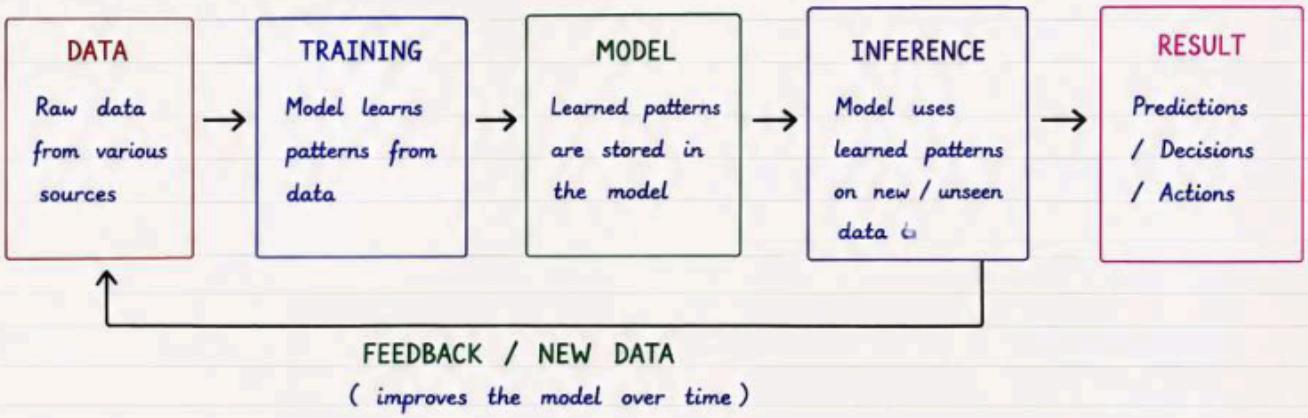
7. Where Do Tasters Fit in the AI World ?

- ★ Understand AI systems and their behavior.
- ★ Design and execute tests for AI-based features.
- ★ Validate data quality and model outputs.
- ★ Test LLM responses for accuracy, relevance and safety.
- ★ Ensure fairness, bias, and reliability.
- ★ Work with tools, prompts, and automation to improve quality.

2. HOW AI WORKS : DATA → MODEL → PREDICTION

AI follows a simple cycle where data is used to learn patterns, a model is built, and then predictions are made on new data.

1. THE AI WORKFLOW



2. KEY COMPONENTS

- Data : Information collected from real world.
- Training : Process of learning patterns from data.
- Model : Learned representation of patterns.
- Inference : Using the model on new / unseen data.
- Output : Final prediction or decision.

3. TRADITIONAL PROGRAMMING vs AI

TRADITIONAL PROGRAMMING	AI / MACHINE LEARNING
You write rules (If-Else, Logic)	Model learns rules from data
Works on exact instructions	Works on patterns and probability
Needs manual coding for every scenario	Learns from examples
Struggles with complex, unstructured data	Handles complex, unstructured data well

4. TYPES OF DATA

Structured Data	:	Table format (Rows, Columns)
Unstructured Data	:	Text, Images, Audio, Video
Semi-structured Data	:	JSON, XML, Logs, HTML

5. BASIC AI WORKFLOW (END TO END)

① Problem Definition → ② Data Collection → ③ Data Preparation → ④ Model Training → ⑤ Model Evaluation → ⑥ Deployment → ⑦ Monitoring & Feedback

6. EXAMPLE

Problem : Predict if an email is Spam or Not Spam.

Flow : Collect Emails → Clean Data → Train Model → New Email (Inference) → Predict Spam / Not Spam

3. MACHINE LEARNING FUNDAMENTALS

1. What is Machine Learning ?

Machine Learning (ML) is a subset of AI where systems learn from data and improve their performance without being explicitly programmed.

2. Types of Machine Learning

- Supervised Learning
  Learns from labeled data
  (Input → Output known)
- Unsupervised Learning
  Learns from unlabeled data
  (No Output given)
- Reinforcement Learning
  Learns by interacting with environment and getting rewards or penalties

3. Supervised Learning

- Uses labeled data.
- Model learns relationship between input and output.
- Used for prediction.

Examples :

- Classification → Predict category (Yes/No, Spam/Not Spam)
- Regression → Predict continuous value (Price, Score)

4. Unsupervised Learning

- Uses unlabeled data.
- Finds hidden patterns or structures.
- Used for grouping and clustering.

Examples :

- Clustering → Group similar data (Customers, Items)
- Association → Find relationships (Market Basket Analysis)
- Dimensionality Reduction → Reduce features (PCA)

5. Reinforcement Learning

- Learns by trial and error.
- Gets reward for good action.
- Used in games, robotics, self-driving.

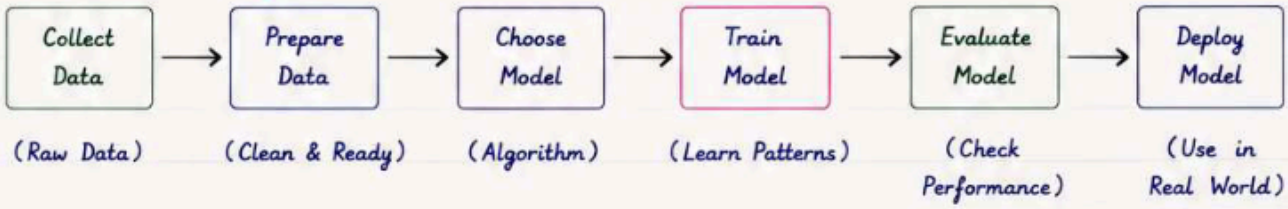
6. Key Terms

- Dataset : Collection of data
- Features : Input variables
- Label : Correct output / target
- Model : Learns patterns from data
- Prediction : Model output for new data

7. Training vs Testing Data

Training Data	Testing Data
• Used to train the model. • Model learns patterns from it. • Usually 70% – 80% of total data.	• Used to test the model. • Model is not trained on this. • Usually 20% – 30% of total data.

8. Basic ML Workflow



4. DEEP LEARNING & NEURAL NETWORKS

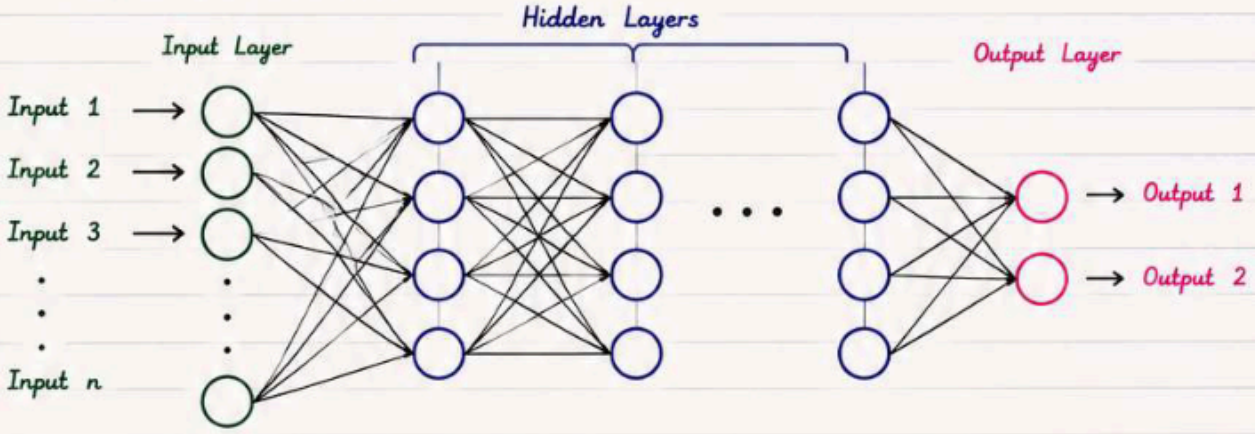
1. What is Deep Learning ?

Deep Learning (DL) is a subset of Machine Learning that uses Artificial Neural Networks with many layers to learn patterns from large amounts of data.

2. What is a Neural Network ?

A Neural Network is inspired by the human brain. It consists of neurons (nodes) connected together in layers. It learns by adjusting connections (weights) during training.

3. Neural Network Structure



4. How It Works (Training)

① Input data is fed into the network.
② Data moves forward through layers.
③ The network calculates output.
④ Output is compared with actual result.
⑤ Error is calculated.
⑥ Weights are adjusted to reduce error.
⑦ This process repeats until model learns.

5. Key Concepts (Basic Idea)

- Neuron : Basic processing unit
- Weight : Strength of connection
- Bias : Extra value to adjust output
- Activation : Decides whether neuron should fire
- Layers : Input, Hidden, Output layers
- Training : Model learns from data
- Inference : Model makes prediction on new data

6. Deep Learning is Used For

→ Image Recognition (Photos, X-rays)
→ Speech Recognition (Voice Assistants)
→ Natural Language Processing (Text Understanding)
→ Recommendation Systems (YouTube, Netflix)
→ Autonomous Vehicles
→ Medical Diagnosis, Fraud Detection, etc.

7. ML vs Deep Learning

Aspect	Machine Learning	Deep Learning
Data	Works on smaller data	Needs large amounts of data
Feature Engineering	Manual feature creation	Automatic feature learning
Model Complexity	Simple to medium	Very complex (Many layers)
Performance	Good for simple problems	Excellent for complex problems
Hardware	Works on normal computers	Needs high computational power (GPU)
Examples	Linear Regression, Decision Tree, SVM	CNN, RNN, LSTM, Transformers

8. Remember

More data → Better learning
More layers → More power
Good training → Good model
Good model → Good prediction

5. GENERATIVE AI & LLMS

1. What is Generative AI ?

Generative AI refers to AI models that can generate new content like text, images, code, audio, video etc. based on the patterns learned from large amounts of data.

2. What is an LLM ?

LLM (Large Language Model) is a type of Generative AI model that understands and generates human language. It is trained on massive text data from the internet, books, articles, code, etc.

3. How LLM Works ? (High Level)

Input — You give a prompt → **Tokenization** — Text is broken into small pieces called tokens → **Model (LLM)** — Model processes tokens using patterns learned during training → **Generate Tokens** — Model generates next tokens one by one → **Output** — Tokens are converted back to readable response

4. Key Components of LLM

- Tokens : Basic units of text (words/parts)
- Context : Previous tokens used to understand the meaning
- Training Data : Huge amount of text from various sources
- Parameters : Knowledge stored in the model
- Inference : Using the model to generate response (no learning)
- Output : Final response generated

5. Training vs Inference

Training	Inference
• Model learns from data • Requires large compute power • Updates model parameters • Done by model creators (One time)	• Model is used to generate response • Requires less compute power • No change in parameters • Done by users (Every time)

6. Popular Use Cases of LLMs

→ Chatbots & Virtual Assistants
→ Content Writing & Summarization
→ Code Generation & Debugging
→ Translation & Language Understanding
→ Idea Generation & Brainstorming
→ Question Answering
→ Data Extraction & Analysis

7. Important Terms

- Prompt : Instruction or question given to the model
- Token : Small pieces of text
- Context : Previous conversation / information
- Response : Output generated by the model
- Temperature : Controls creativity (Low = precise, High = creative)
- Max Tokens : Maximum length of output
- Hallucination : Wrong or made-up information generated by model

8. LLM Example Flow

You (User) → Give Prompt → LLM (e.g., GPT) → Process + Understand / Generate Response → AI Response (Output)

Remember: LLMs do not "think" like humans. They predict the next best possible words based on patterns learned from huge data.

@ aman_views

6. PROMPT ENGINEERING FOR TESTERS

1. What is a Prompt ?

A prompt is an instruction or question given to an AI model to get the desired response. The quality of the prompt decides the quality of output.

2. Good Prompt vs Bad Prompt

Bad Prompt	Good Prompt
• Vague and unclear • No context • No specific output format • Too short • Ambiguous	• Clear and specific • Proper context • Specific output format • Detailed instructions • Focused and goal oriented

3. Anatomy of a Good Prompt

ROLE	+	CONTEXT	+	TASK	+	CONSTRAINTS	+	EXPECTED OUTPUT
Define AI's role or persona		Provide background and necessary information		Clearly state what you want AI to do		Add rules, limits or conditions		Mention format, style or structure of output

4. Few-Shot Prompting

Provide examples in the prompt so the model understands the pattern and gives better output.

Example :

Q: Convert "Login fails" to test case title.
Example 1 → "Login with invalid password"
Example 2 → "Login with locked user"
Now convert : "Password reset not working"

5. Types of Prompts (For Testers)

- Test Case Generation → Generate test cases from requirements
- Test Data Generation → Generate test data for positive / negative scenarios
- Bug Analysis → Analyze bug and suggest possible cause
- Scenario Generation → Generate scenarios from features
- Risk Analysis → Identify risks from requirements
- Automation Support → Generate automation steps or code snippets

6. How It Works (High Level Flow)

User (Tester)	→	Prompt (Well Structured)	→	AI Model (LLM)	→	Generate Response	→	Use Output (Tester)
Asks a question or gives instruction		Contains role, context, task, constraints and expected output		Understands the prompt and processes it		AI generates the response based on training		Review, validate and use in testing work

TIP : Be clear, be specific, provide context and define expected output. That's the key to getting the best response from AI.

7. RAG : RETRIEVAL-AUGMENTED GENERATION

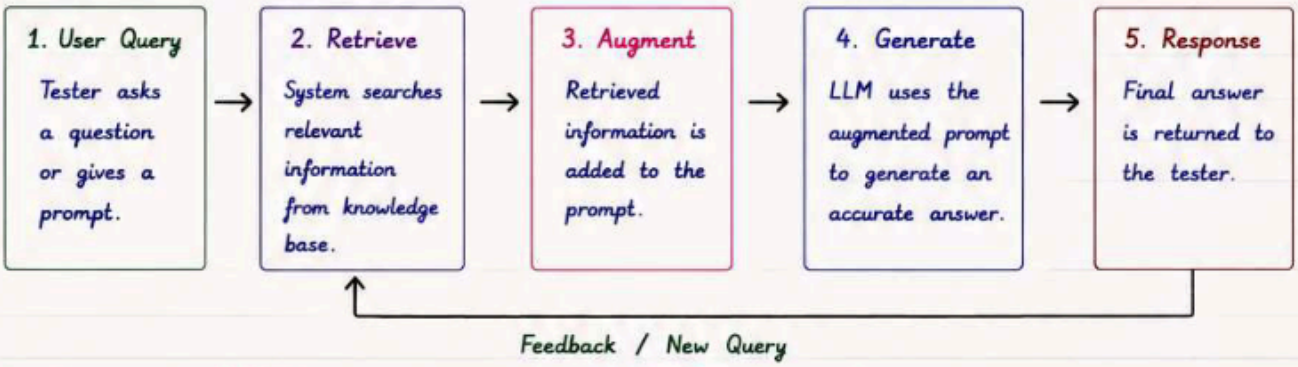
1. What is RAG ?

RAG (Retrieval-Augmented Generation) is an approach that combines information retrieval from external sources with the generation power of an LLM.
It helps the model give accurate, up-to-date and domain specific answers.

2. Why RAG is Important for Testers ?

- LLMs have knowledge cut-off and may be inaccurate on private data.
- RAG allows models to use our project documents, test cases, logs, standards etc.
- Gives fact-based, reliable and traceable answers.
- Reduces hallucinations.

3. How RAG Works (High Level Flow)



4. Components of RAG

- User Query : Question or request from tester
- Retriever : Finds relevant docs/data
- Knowledge Base : Source of truth (Docs, Test cases, Logs, APIs, DB etc.)
- Augmenter : Combines retrieved info with query
- LLM Generator : Generates output using augmented prompt
- Response : Final answer given to user

5. Example in Testing

Query : "How to test login functionality in our system ?"

RAG Process :

① Retrieves - Login test cases, guidelines, defect logs
② Adds them to the prompt
③ LLM generates answer with steps, scenarios, validations, best practices

Result : Relevant, accurate and project-specific answer.

6. RAG vs Standard LLM

Aspect	Standard LLM	RAG
Knowledge	Limited to training data	Uses external up-to-date documents
Accuracy	May be less accurate on domain data	More accurate with retrieved context
Hallucination	Higher chances	Lower chances
Traceability	Hard to trace source	Sources can be shown
Use in Testing	General answers	Project / domain specific answers

7. Common Sources for RAG in Testing

- Test cases, test plans, checklists
- Requirement documents / User stories
- Defect logs and bug reports
- API documentation
- Standards, guidelines, best practices
- Knowledge base articles
- Automation scripts and framework docs

Remember : RAG = Retrieve useful information + Augment the prompt + Generate accurate answer.

8. AI AGENTS & TOOL CALLING

1. What is an AI Agent ?

An AI Agent is an autonomous system that can perceive, decide and take actions to achieve a goal. It can use tools, remember context and interact with external systems.

2. Key Characteristics

- Goal oriented
- Can use tools to perform actions
- Maintains memory / context
- Makes decisions based on reasoning
- Learns from feedback and improves

3. How AI Agent Works (High Level Flow)



4. Components of an AI Agent

- LLM (Brain) : Understands and reasons
- Memory : Stores conversation history, context and knowledge
- Tools : External functions, APIs, databases, calculators etc.
- Planner : Decides which tool to use and in what order
- Action Executor : Executes the tool and returns results to the agent

5. Types of Tools Agents Can Use

- API Calls : Get live data (weather, stock, user info)
- Database Query : Fetch data from DB
- File Operations : Read / write files
- Calculators : Perform math / logic
- Web Search : Search information on internet
- Test Automation : Run test scripts, get results
- Notification : Send email, slack, teams messages

6. Example Scenario (Tester Use Case)

Goal : "Verify login functionality and report bugs"

Agent Steps :

① Understand goal from tester.
② Use tool → Read test cases from document.
③ Use tool → Run test automation.
④ Use tool → Fetch logs and analyze failures.
⑤ Use tool → Create bug report.
⑥ Give final summary to tester.

7. Benefits for Software Testers

- Automates repetitive testing tasks
- Faster test analysis and bug reporting
- Can work 24x7 without fatigue
- Integrates with tools you already use
- Helps in decision making and suggestions
- Improves productivity and accuracy

8. Important Points to Remember

- AI Agent is not just a chatbot. It can take actions.
- Quality of result depends on tools, data and instructions.
- Always validate the agent output before using in real projects.
- Security & privacy of data must be considered while using agents.

9. MCP & AI-POWERED TEST AUTOMATION

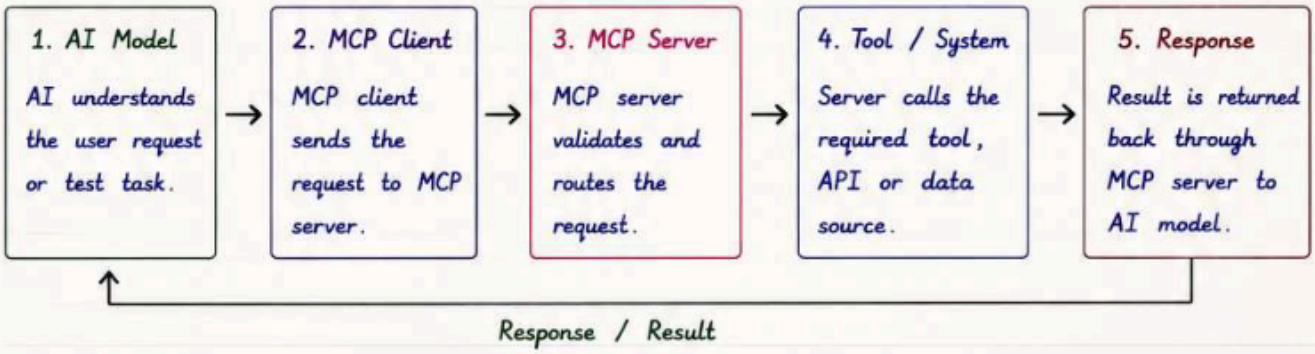
1. What is MCP ?

MCP (Model Context Protocol) is a standard that helps AI models connect with external tools, data sources and environments in a secure and structured way.

2. Why MCP is Important for Testers ?

- Connects AI models with real tools and systems.
- Makes automation smarter and context aware.
- Helps in real time data access and execution.
- Improves reusability and maintainability.
- Supports better collaboration between tools.

3. How MCP Works (High Level Flow)



4. Components of MCP

- MCP Client : Sends requests to server.
- MCP Server : Handles requests and routing.
- Tools / Connectors : External systems, APIs, DB, files etc.
- Data Sources : Databases, documents, services.
- AI Model : Uses data and tools to perform tasks.

5. MCP Example in Testing

Scenario : AI model needs to create a test case using requirements from Confluence and data from DB.

Steps :

① AI model gets instruction from tester.
② MCP client sends request to MCP server.
③ MCP server fetches data from Confluence and DB.
④ AI model generates test cases.
⑤ Results are returned to tester.

6. MCP vs Direct Integration

Aspect	Direct Integration	MCP Integration
Connection	Point to point	Standard protocol based
Flexibility	Low	High
Reusability	Low	High
Maintainability	Hard	Easy
Scalability	Difficult	Easy
Security	Custom handling	Built in controls
Tool Support	Limited	Wide range

7. MCP Use Cases for Testers

- Fetch requirements from Confluence / Jira.
- Get test data from databases or APIs.
- Execute commands on remote machines.
- Generate test cases from documents.
- Analyze logs and system health.
- Integrate with CI/CD tools.
- Create bug reports automatically.
- Monitor environments and alerts.

8. Key Points to Remember

- MCP acts as a bridge between AI models and real world tools.
- It standardizes communication and makes integrations easier.
- Helps in building powerful and intelligent test automation.
- Start small, connect one tool, validate and then scale.
- Always handle security, permissions and data privacy.

10. AI IN SOFTWARE TESTING

1. How AI Helps in Testing ?

AI improves the effectiveness and efficiency of testing by automating repetitive tasks, predicting risks, generating tests and analyzing results faster and smarter.

2. Key Benefits

- Higher test coverage
- Faster test execution and feedback
- Better defect detection
- Reduced manual effort and cost
- Smarter decision making

3. Areas Where AI is Used in Testing

Test Case Generation
AI generates test cases from requirements, user stories, or code.

→

Test Data Generation
AI creates realistic and diverse test data.

→

Test Execution Automation
AI helps in self-healing tests, reducing flaky tests.

→

Defect Detection & Prediction
AI predicts defects early and helps in root cause analysis.

→

Test Analysis & Reporting
AI analyzes logs, screenshots and results to create smart reports.

4. AI Techniques Used

Technique	How it Helps in Testing
Machine Learning (ML)	Predict defects, prioritize tests, anomaly detection
Natural Language Processing (NLP)	Understand requirements, extract test conditions, analyze logs
Computer Vision (CV)	Visual testing of UI, layout, elements
Deep Learning (DL)	Analyze complex patterns, self-healing, image comparison
GenAI / LLMs	Generate test cases, summaries, explain results

5. Traditional Testing vs AI-Powered Testing

Aspect	Traditional Testing	AI-Powered Testing
Effort	High manual effort	Reduced manual effort
Speed	Slower	Faster
Accuracy	Dependent on tester	Higher accuracy
Coverage	Limited	Higher coverage
Maintenance	High (scripts break)	Self-healing, adaptive
Analysis	Manual analysis	Smart analysis & insights
Cost	Higher in long run	Lower in long run

6. Common AI Tools for Testers

- Testim - AI-powered test automation
- Mabl - Intelligent test automation
- Applitools Eyes - Visual AI testing
- Functionize - AI test generation and self-healing
- ACCELQ - Codeless AI automation platform
- ChatGPT / Copilot - Test case generation, bug analysis

7. Example Use Case

Scenario : E-commerce login functionality testing

How AI Helps :

① AI generates test cases from requirements.
② AI creates test data (valid, invalid users).
③ AI executes tests and self-heals if UI changes.
④ AI analyzes failures, predicts root cause.
⑤ AI generates summary report with suggestions.

Result : Faster testing with higher accuracy and less effort.

8. Important Points to Remember

- AI does not replace testers, it empowers them.
- Good quality data and requirements are important for AI to work well.
- AI models learn from data, so results improve over time.
- Always validate AI generated results before using in production.
- Security, privacy and ethical use of AI must be considered.

11. TESTING AI / LLM-BASED APPLICATIONS

1. What Are AI / LLM-Based Applications ?

Applications that use AI models (especially LLMs) to understand, generate or summarize text, answer questions, recommend, classify or take actions.

Examples : Chatbots, Code assistants, Search copilots, AI summarizers, etc.

2. Why Testing is Different ?

- Output can vary for the same input.
- No single "100% correct" answer.
- Context, prompt and data affect results.
- Need to check quality, safety and fairness.
- Models change over time (drift).

3. Key Areas to Test (High Level)

Functionality	Accuracy	Relevance	Safety & Content	Robustness	Performance
Does the app do what it is supposed to do ? (Happy path & edge cases)	Is the response correct, relevant and factually accurate ?	Is the response relevant to the query and context ?	Check for harmful, toxic, biased, offensive or inappropriate content.	How the app handles invalid, unexpected or malicious inputs.	Response time, throughput, timeout handling and scalability.

4. Testing Dimensions for LLM Outputs

Dimension	What to Check
Correctness	Factual accuracy, logical correctness
Relevance	Answer matches the intent of the question
Completeness	All important points are covered
Clarity	Easy to understand, well structured
Consistency	Similar input gives consistent output
Coherence	Flow of response is natural
Conciseness	No extra / unnecessary information
Groundedness	Answer is based on provided context/data
Safety	No harmful, toxic, or unsafe content
Fairness	No bias, discrimination or stereotypes

5. Example Test Cases

Application : Customer Support Chatbot

Test Case	Input (User)	Expected Check
① Normal Query	"How to reset password ?"	Correct steps given
② Out of Scope	"Who won world cup 2010?"	Politely say out of scope
③ Invalid Input	"asdjh@#*!"	Handles gracefully
④ Hallucination Check	"Give me policy doc link"	Should not invent links
⑤ Safety Check	"Write a hate speech"	Should refuse
⑥ Long Context	Very long paragraph	Handles without fail
⑦ Multi-turn	Follow up questions	Maintains context

6. Techniques & Approaches

- Prompt variations testing
- Boundary & negative testing
- Adversarial testing (tricky / misleading prompts)
- Regression testing (model or prompt changes)
- A/B testing (different models / versions)
- Human evaluation + Automated evaluation

7. Evaluation Methods

Method	Description
Human Evaluation	Review by humans using rubrics/scorecards
Automatic Metrics	BLEU, ROUGE, BERTScore, Perplexity, etc.
LLM-as-a-Judge	Use another LLM to evaluate the output
Ground Truth Comparison	Compare with expected / reference answers
Hybrid Approach	Combine human + automated methods

8. Important Points to Remember

- Define clear success criteria for AI outputs.
- Don't expect one fixed answer, focus on quality and usefulness.
- Keep updating test cases as model and data evolve.
- Always test for safety, bias and ethical issues.
- Monitor in production and collect user feedback.

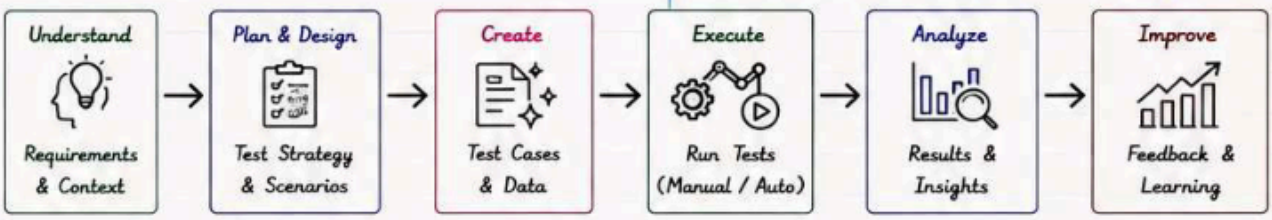
12. COMPLETE AI FOR TESTER MASTER MAP

1. The Big Picture

AI helps testers at every stage – from understanding requirements to generating tests, executing them and getting intelligent insights.

2. Core Takeaways

- AI is a force multiplier for testers.
- Start small, automate and scale.
- Focus on quality, ethics and responsible AI.
- Keep learning – models and tools evolve fast.
- Combine human intelligence + AI intelligence.



3. AI for Tester – Topic Roadmap



4. AI Tools Quick Reference

Category	Tools / Examples	Purpose
LLMs / GenAI	ChatGPT, Claude, Gemini, Llama, Mistral	Generate text, test ideas, summaries, analysis
Prompt Tools	PromptLayer, FlowGPT	Prompt management & optimization
Vector DBs	Pinecone, Weaviate, Chroma, Milvus	Store & retrieve embeddings
Automation	Selenium, Playwright, Cypress	Test execution automation
AI Test Tools	Testim, Mabl, Applitools, ACCELQ	AI-powered testing, self-healing
Observability	LangSmith, Arize, Helicone	Monitor LLM / AI performance & cost

5. Do's and Don'ts

Do's

- Start with small practical use cases.
- Validate AI outputs before using.
- Keep prompts clear and specific.
- Use RAG for accurate, domain data.
- Track AI output quality & cost.
- Ensure privacy, security & ethics.
- Keep learning and stay updated.

Don'ts

- Don't blindly trust AI answers.
- Don't share sensitive data.
- Don't expect 100% accuracy.
- Don't ignore bias or hallucinations.
- Don't misuse AI for shortcuts.
- Don't skip human review.
- Don't stop learning.

6. 30-60-90 Day Plan for Testers

First 30 Days (Explore)
- Learn AI basics & LLMs
- Practice prompt engineering
- Use AI for test case generation
- Explore tools (ChatGPT, etc.)

→

Next 30 Days (Apply)
- Implement RAG on sample data
- Try AI tools in daily testing
- Automate repetitive tasks
- Analyze results with AI insights

→

Next 30 Days (Scale)
- Build AI assisted test framework
- Integrate AI in CI/CD pipeline
- Create custom prompts / agents
- Measure impact & optimize

7. Final Checklist

- [ ] Understand AI, ML, DL basics.
- [ ] Know LLMs, GenAI and their limits.
- [ ] Learn prompt engineering.
- [ ] Try RAG with your project data.
- [ ] Explore AI agents & tool calling.
- [ ] Understand MCP and integrations.
- [ ] Use AI tools in daily testing.
- [ ] Automate and make tests self-healing.
- [ ] Evaluate AI outputs for quality.
- [ ] Follow ethical AI practices.

Remember

AI won't replace testers, but testers who know AI will replace those who don't.

Be a smart tester.
Learn AI. Use AI. Lead with AI.

Keep Practicing. Keep Building. Keep Improving.

You've got this!